



Demo Script

About 300 words spoken · slide 3 for the architecture, then the live dashboard and the AWS CloudWatch graph

Before recording: press RESET on the dashboard · confirm METRICS_WITH_AWS: ON · open the CloudWatch graph (15-minute view) in another tab · have slide 3 ready · record straight through and **cut the 1–3 minute patch wait** in editing.

TIME	BEAT · ON SCREEN	SAY
0:00–0:15	Problem Dashboard: logo and title animating	HP Wolf Security research shows cheap, AI-built attacks are beating static defenses. Most tools just block, log, and forget, so the attacker tweaks the attack and tries again. NanoPot is for the security engineer who is done with that: a honeypot on one HP ZGX Nano that turns every attack into a tested patch.
0:15–0:38	Architecture Slide 3 · point along the arrows: top diagram, then bottom	[top] Every request is checked by Jev , a cloud decision AI, and by our own model on the Nano . Normal users reach the real app; known attacks are blocked; new attacks are redirected to a decoy where an 80-billion-parameter AI invents fake data. [bottom] Meanwhile the Nano learns: one AI writes a fix, four tests check it, a second AI tries to break it, and approved fixes go into Redis and the app's code.
0:38–0:46	Normal user Dashboard · click Normal_Search	A normal customer: Jev and our model both say safe, so it goes straight to the real shop.
0:46–1:05	Attack → decoy SEARCH · type <code>shoes; cat / etc/passwd</code> → SEND · point at the terminal, then HACKER'S_POV	Now an attacker. Jev flags command injection, which our small model alone missed. Instead of blocking, we redirect them to the decoy: fake data, with fake passwords unique to them . They think they've won.
1:05–1:18	Learning The fight in AUTO_PATCH_ENGINE → cut → K.O. → click the patch card	While they're busy, NanoPot fights back. The fix is written, tested... approved . There's the rule, now written into the app's own code .
1:18–1:30	Blocked Send the same attack → FIREWALL BLOCKED · Auth_Bypass , then Use_Stolen_Creds → ACCOUNT COMPROMISED	Same attack again: blocked instantly . And if they try the password they stole: caught, and traced to their session .
1:30–1:52	Cloud comparison AWS CloudWatch graph (15-minute view) · point at each line	Every AI call also ran on AWS Bedrock , same model, same prompt. This is Amazon's own dashboard. [green] Tokens we send, [orange] tokens back: that's the cloud bill . [blue] Response time, slightly faster than the Nano. [red] Far below AWS's usage limits. Amazon's counts match ours call for call , and the Nano does the same work 38 to 59 times cheaper , with attack data staying on-site.
1:52–2:00	Close Back to the dashboard	Attackers get fool's gold. We get the fix. NanoPot : detection, deception and self-healing, on one HP ZGX Nano.

Delivery

- Rehearse once with a timer; the problem statement and slide 3 are the densest parts.
- On slide 3, point along the arrows instead of reading the boxes.
- On the CloudWatch graph, hover a peak so AWS shows the exact values.

If something goes off-script

- **Attack blocked on the first try:** it was patched in a rehearsal. Press RESET and redo it.
- **Terminal says JEV: unavailable:** "the Nano's model decides on its own, that's our fallback."
- **"Nothing stolen yet":** click Auth_Bypass before Use_Stolen_Creds.